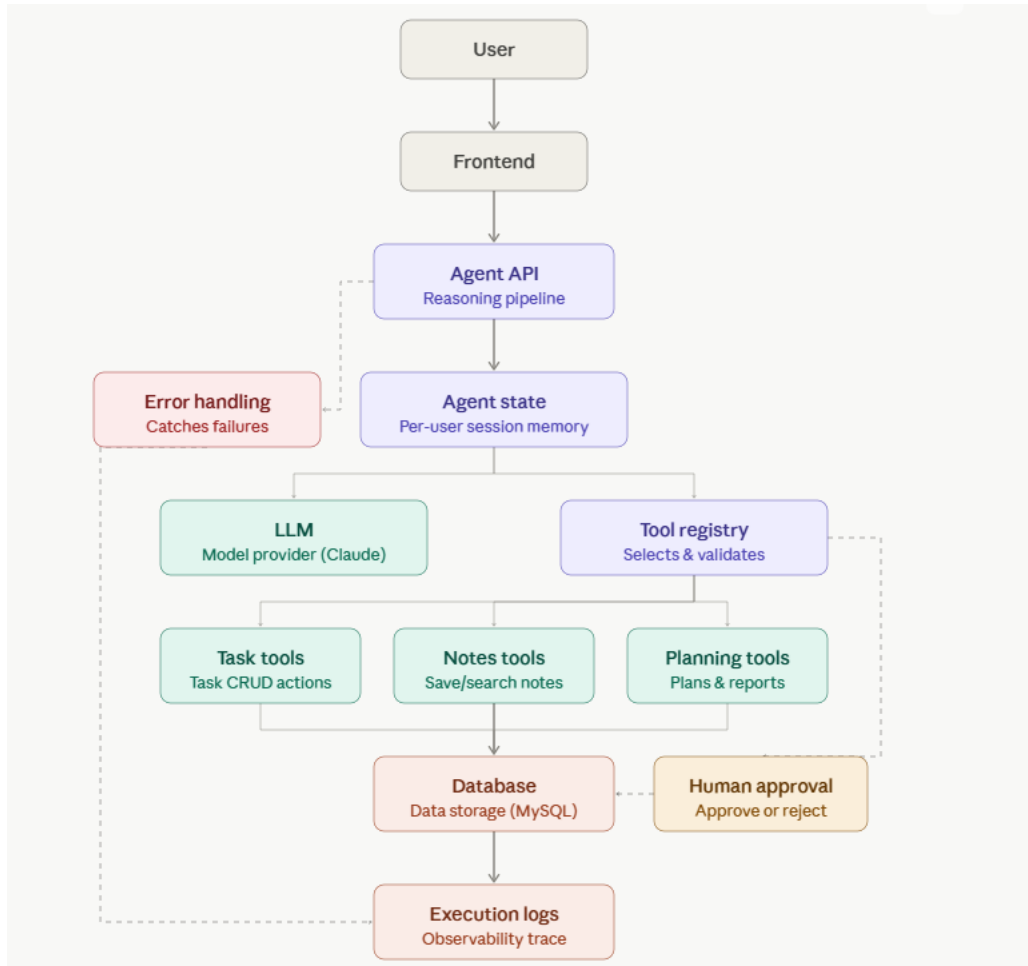




Component explanations

User : The person interacting with the system through natural-language requests, task management, or reviewing plans and logs.

Frontend : The Bootstrap/vanilla JS interface (chat, task board, plans, approvals, logs pages). Sends requests to the Agent API and renders responses, execution traces, and approval prompts.

Agent API : The Flask entry point (`agent_routes.py`) that receives requests and hands them to the controller, which orchestrates the full reasoning pipeline: intent analysis → validation → tool execution → response generation.

Agent state (session memory) : Per-user/session context tracked across turns (browser-scoped via `localStorage` session keys, per the "Known Limitations" section). This is what lets the agent remember what was just discussed without a full login system.

LLM (model provider) : Claude, accessed via the OpenRouter API (`llm_service.py`). Used for intent classification, tool selection reasoning, task extraction from unstructured notes, summarization, and email drafting.

Tool registry : The central dispatch (`tool_registry.py`) that holds all 12+ registered tools, exposes their schemas to the LLM for selection, and routes a chosen tool call to the right module after `validator.py` checks its inputs.

Task / Notes / Planning tools : The three tool modules doing the actual work: task CRUD operations, note saving/searching, and daily/weekly plan + productivity report generation. (Email drafting and priority analysis also live at this layer, omitted from the diagram for space — they're peers of these three.)

Human approval : The gate specifically in front of `complete_task` and `delete_task`. When the tool registry selects one of these, execution pauses, an `ApprovalRequest` record is created, and the action only proceeds to the database once the user explicitly approves or rejects it via the Approvals page.

Error handling : Cross-cutting rather than a single module: input validation failures (`validator.py`), invalid/incomplete requests, and tool execution errors are caught at the controller level, converted into graceful natural-language responses instead of crashes, and recorded in the execution log rather than silently failing.

Database : MySQL, accessed via SQLAlchemy/PyMySQL, storing `Task`, `Note`, `Plan`, `ExecutionLog`, `ApprovalRequest`, and `AgentMessage` records. This is the single source of truth the tools read from and write to.

Execution logs (observability) : Every decision step — which intent was inferred, which tool was chosen, whether approval was required, what the result was — is written by

`execution_logger.py` and viewable in the Logs page, giving full traceability of agent behavior for debugging and evaluation.